

NERD: A Node Editor for Csound Integration in Scientific and Artistic Contexts

Lovre Bogdanic*

Max Planck Institute for Empirical Aesthetics, Frankfurt am Main, Germany
lovre.bogdanic@ae.mpg.de

Abstract. NERD (Node Editor for Real-Time DSP) is an open-source, Python-based visual programming environment that organises real-time digital signal processing pipelines as networks of heterogeneous, data-linked nodes on an interactive canvas. Its central feature is a first-class integration with Csound: synthesis instruments authored as standard `.csd` files are incorporated into any pipeline by adding inline comment annotations that declare their input and output channels; NERD’s meta-parser automatically exposes these channels as typed node ports without requiring any modification to the Csound instrument logic. Surrounding infrastructure—biosignal acquisition, motion tracking, MIDI, OSC, and LabStreamingLayer network streaming—is handled by a library of plugin nodes backed by a daemon-thread worker model. NERD originated in a Volkswagen-Foundation funded research project on musical improvisation and joint action. It has since been used in several scientific and artistic events, including the Science Festival Frankfurt 2024 and the neuroscience outreach event Frankfurt hat Hirn 2026, and a live heart rate sonification installation during an artist-in-residence program in 2025. The software is released under the GNU General Public License (v3) and is available at <https://gitlab.gwdg.de/lovre.bogdanic/nerd>.

Keywords: node editor, real-time DSP, Csound integration, biosignal sonification, Python, live performance

1 Introduction

Real-time audio applications that combine scientific data acquisition with synthesis engines are growing in complexity: a single pipeline may involve multichannel biosignal streams, computer-vision trackers, network clock synchronisation, and multiple Csound instruments, all running concurrently. Building such setups typically demands either substantial bespoke Python glue code or adoption of general-purpose visual programming environments such as Max/MSP [1] or Pure Data [2], neither of which is natively compatible with the Python-centric scientific software stack.

NERD (Node Editor for Real-Time DSP) was designed to fill this gap. It adopts a Csound-first philosophy: all synthesis audio-rate signal processing are delegated to Csound instruments, while NERD provides the surrounding infrastructure as a visual canvas of connected nodes. The tool targets researchers and musicians who are familiar with Csound but need a structured, reusable, and reproducible way to manage the non-audio parts of a complex pipeline.

Section 2 describes the system architecture; Section 3 details the Csound integration mechanism; Section 4 reports three deployments across two use cases; Section 5 concludes with the lessons learned and planned extensions.

2 System Architecture

NERD is written in Python (3.10–3.12) and uses DearPyGui [3] as its GUI framework, which provides a GPU-accelerated, retained-mode node editor widget. The result is a lightweight, cross-platform desktop application.

* This work was carried out within the project *Creative musical dialogues between human and machine: a novel approach to studying improvisation and joint action*, funded by the Volkswagen Foundation (PIs: Dr. Örjan de Manzano, Max Planck Institute for Empirical Aesthetics; Dr. Oded Ben-Tal, Kingston University London).

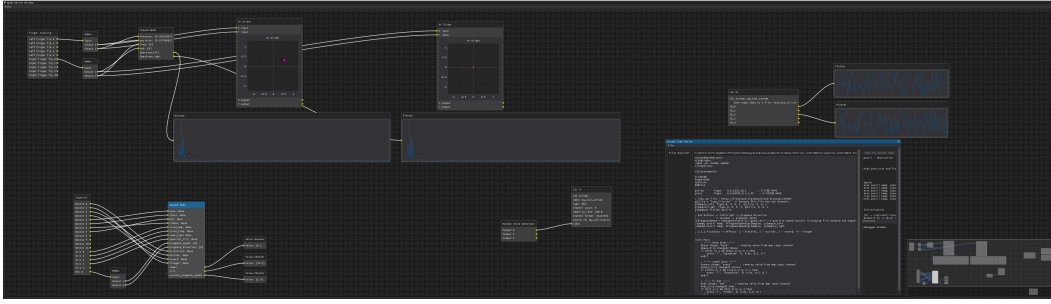


Fig. 1. The NERD canvas. Rectangular nodes are connected by typed links that carry `numpy.ndarray` payloads. The built-in node library covers sensor acquisition, buffering, network streaming (LSL, OSC), signal processing, MIDI, and Csound synthesis; each `CsoundNode` port is declared by a single comment annotation in the `.csd` file.

2.1 Node and Attribute Model

The fundamental unit is the *Node*: a rectangular canvas element that encapsulates a single processing stage. Each node exposes typed *NodeAttributes*—output attributes that push data downstream and input attributes that receive it. All payload data is represented as `numpy.ndarray` objects, enabling direct interoperability with the scientific Python ecosystem. Links connect an output attribute to one or more input attributes; calling `set_data()` on an output propagates the array synchronously to all linked inputs, invoking each input’s `execute()` callback. Fig. 1 shows an example project with *Finger-Tracking* and *Joystick* instruments.

The attribute type system controls the DearPyGui widget rendered inside the node (float slider, integer input, signal plotter, or a plain label socket for routing-only connections) and the data-type contract at the link endpoint. At the moment of writing, twenty-five built-in node classes are organised into six categories: general-purpose controls (sliders, buttons, plotters, loggers, de-multiplexers, data generators), DSP utilities, circular buffering, network I/O (LSL, OSC, NTP synchronisation), and plugin nodes for Csound, MIDI, joystick, camera/MediaPipe hand tracking, ECG processing, and piano-vision MIDI.

2.2 Worker Thread Model

Nodes that produce data continuously—hardware-sensor readers, file-based generators, or network-stream consumers—run a daemon thread controlled by a `_running` Boolean flag. When a node is removed or a project is closed, the flag is cleared and the thread is joined. DearPyGui calls are never made from worker threads; computed results enter the node graph exclusively through the thread-safe `set_data()` path.

For inputs whose data originates outside the DearPyGui callback chain, a `queue.Queue`-backed buffer is provided in `CrossThreadInputAttribute` class. That buffer drains to the most recent chunk on each read, avoiding unbounded accumulation when the consumer is transiently slower than the producer.

2.3 Project Persistence and Plugin System

A project is serialised as newline-delimited JSON, one record per node, containing each node’s Python class path, label, canvas position, attribute UUIDs, and custom configuration fields. Link topology is reconstructed from the saved UUIDs on load. This human-readable format supports version control and manual inspection of experimental setups.

New node types are registered by appending an entry to `registered_nodes.json`, mapping a human-readable name to a dotted Python import path. A `NodeRegister` class dynamically imports each class at startup; nodes whose optional dependencies are absent are logged as warnings and skipped, leaving the remainder of the project functional on minimal installations.

Table 1. Meta-opcode annotations and the NodeAttribute type each creates.

Annotation opcode	Direction	NodeAttribute type
<code>in_chn</code>	NERD→Csound Input	(scalar channel)
<code>in_chn_array</code>	NERD→Csound Input	(array)
<code>out_chn</code>	Csound→NERD Output	(scalar channel)
<code>out_table</code>	Csound→NERD Output	(function table)

3 Csound Integration

The `CsoundNode` class wraps a standard `.csd` file and runs Csound in-process via the `ctcsound` Python API. When a file is loaded, `CsoundMetaParser` scans the `<CsInstruments>` block for line comments of the form:

```
; <@nerd: opcode,channel_name[,table_number]>
```

Four opcodes are supported (Table 1).

Each annotation creates a node port labelled with `channel_name`. Inside the `.csd`, the instrument uses standard `chnget` and `chnset` opcodes to communicate over those channels; the annotations are ordinary line comments and are therefore invisible to the Csound preprocessor. An existing `.csd` can be integrated into any NERD project by adding a single annotation comment per channel, leaving all synthesis logic untouched. The following excerpt from the heart rate sonification instrument demonstrates the pattern:

```
instr ScheduleSamples
  kvalue chnget "value" ; <@nerd: in_chn,value>
  kInterval = kvalue / 60
  kTrig      metro kInterval
  if (kTrig == 1) then
    event "i", "PlaySample", 0, 3
  endif
endin
```

For `out_table` annotations, a worker thread polls the Csound function table at every audio callback period, copies it to a NumPy array, and calls `set_data()`, making spectral data from real-time FFT analysis available to downstream `Plotter` nodes without any additional bridging code.

4 Use Cases

4.1 Finger-Tracking and Joystick Instruments – Science Festival Frankfurt 2024 / Frankfurt hat Hirn 2026

NERD has been used at two public-facing events to present interactive synthesis systems driven by physical gesture: the Science Festival Frankfurt 2024 and the neuroscience outreach event Frankfurt hat Hirn 2026. Two complementary computer music instruments ran from the same NERD project file.

Both instruments were designed as interactive demonstrations for general audiences rather than for research use, prioritising immediate engagement over precision control.

The finger-tracking instrument maps the normalised screen coordinates of the index fingertip of each hand—tracked by a `HandTracking` node based on MediaPipe Hands [6]—to four `in_chn` ports of a `CsoundNode`. The left hand controls spatial parameters: horizontal position drives stereo panning and vertical position (interpreted as proximity distance) scales a `doppler` simulation and acts as an amplitude gate that silences the output when the hand is too close to the camera. The right hand controls FM synthesis: horizontal position sets the carrier frequency in the range 110–210 Hz and vertical position scales the modulation index. All four control streams are smoothed with a 10-coefficient `filter2` moving average before use. The stereo output is simultaneously analysed by a real-time FFT pipeline; per-frame magnitude spectra are written to two Csound function tables and

exposed as `out_table` ports, which NERD routes to two `Plotter` nodes for audio-synchronised visual feedback.

The joystick instrument maps a standard joystick to three concurrent synthesis layers inside the same `CsoundNode`. The four shoulder buttons trigger one-shot drum sounds—bass drum, snare, hi-hat, and cymbal—each implemented as a self-contained sub-instrument. The analogue axes control an `fmvoice` formant synthesizer, with separate channels for fundamental frequency, amplitude, and a phoneme index that selects one of 17 `fmvoice` vowel presets. The hat switch adjusts the playback direction and speed of a looped audio sample via `poscil`, allowing the performer to scrub and reverse the playback in real time. The four face buttons toggle audio effects into the signal chain—distortion, exciter, reverb, and flanger—independently and in any combination.

4.2 Heart Rate Sonification – Artist-in-Residence 2025

During a 2025 artist-in-residence programme, NERD powered a live heart rate sonification installation. ECG data measured with a Polar sensor was transmitted over a `LabStreamingLayer` (LSL) [4]. An `LslRx` node received the stream; an `SC Buffer` node windowed the incoming signal; and the `EcgProcessing` node (implemented using the `BioSPPy` library [5]) extracted instantaneous heart rate in beats per minute. The scalar was connected to the `value` input of a `CsoundNode` running the instrument from Section 3, which triggered a recorded heartbeat sample at the computed metronome rate. The complete pipeline required no custom Python code beyond the pre-existing node classes.

5 Discussion and Conclusion

NERD occupies a niche that neither general-purpose visual environments nor scientific Python libraries fully address: a Python-native, Csound-first workflow for mixed scientific and artistic real-time audio applications. The three deployments demonstrate that the same architectural primitives—sensor input, network streaming, buffering, parameter routing, Csound synthesis, and CSV logging—generalise across contexts as different as a public science festival installation, a biosignal-driven artistic sonification and a reproducible scientific study setup.

The deployments validated several design properties that proved practically valuable in the field. The plain-text JSON project format enabled easy manual editing, sharing, and version control of experimental setups. The plugin registry’s dependency-isolation mechanism ensured that a machine lacking, say, the `MediaPipe` package could still open and run the heart rate sonification project unchanged—absent nodes are skipped with a logged warning rather than causing a startup failure. Perhaps most consequentially, the Csound annotation mechanism demonstrated genuine non-invasiveness: all three `.csd` files used across the deployments remained fully valid standalone Csound instruments, runnable directly with the Csound command-line tool independently of NERD. This contrasts with integration approaches that require modifying instrument logic or introducing a proprietary API, and means that existing instrument libraries can be incorporated into a NERD pipeline with minimal friction and without sacrificing portability.

Planned extensions include better integration of NTP-based synchronisation and an expanded library of general-purpose nodes targeting common scientific and artistic tasks.

References

1. Puckette, M.: The patcher. In: Proceedings of the International Computer Music Conference (ICMC), pp. 420–429. Computer Music Association (1988)
2. Puckette, M.: Pure Data. In: Proceedings of the International Computer Music Conference (ICMC), pp. 224–227. International Computer Music Association (1996)
3. Hoffstadt, J.: DearPyGui: A GPU Accelerated Python GUI Framework. GitHub repository (2021). <https://github.com/hoffstadt/DearPyGui>
4. Kothe, C.: Lab Streaming Layer (LSL). GitHub repository (2014). <https://github.com/sccn/labstreaminglayer>
5. Carreiras, C., et al.: BioSPPy: Biosignal Processing in Python. GitHub repository (2015). <https://github.com/PIA-Group/BioSPPy>
6. Lugaresi, C., et al.: MediaPipe: A framework for building perception pipelines. arXiv preprint arXiv:1906.08172 (2019)